

技术报告：语义通信助手

> 赛道一：Agent 应用开发（OpenTrek 平台）

> 最后更新：2026-08-23

1. 系统概述

语义通信助手是在东南大学 OpenTrek Agent 开发平台上构建的领域文献问答 Agent。

系统以 10 篇语义通信核心论文构成领域知识库，提供两条可运行的产品形态：

- B 路线（主推）：`语义通信知识问答` —— 确定性工作流 RAG，
输入问题 → 多轮改写 → 知识库文档召回 → 结果总结 → 带引用的完整回答；
- A 路线（备选）：`语义通信自主Agent3` —— 角色对话型自主 Agent（AutoBotV2/ReAct），
负责自由对话、观点组织与规划式回答。

2. 平台资产清单

2.1 Agent

	agentCode	agentVersion	引擎	状态
识问答（B，主）	`fcae0115-ea0f-4c86-b688-56ac9c88db05`	`1787495161508`	processAgentEngine / workflow	ONLINE，引用校验 + 防编造，e
主Agent3（A）	`407be567-2e47-4391-8249-729d943d1753`	`1787370379269`	processAgentEngine / AutoBotV2 + ReAct	ONLINE，自主问答验证通过

2.2 知识库

名称	kbCode	内容	状态
语义通信知识库	`ryekr4gqw2pn`	10 篇语义通信核心论文	10/10 解析成功（state=200）

文件清单与 chunk 数：

论文	arXiv	chunk 数
DeepSC（Xie et al., 2020）	2006.10685	已解析
Deep JSCC for Wireless Image Transmission	1809.01733	41
Semantic Communications: Principles and Challenges	2201.01389	57
A Theory of Semantic Communication	2212.01485	已解析
Semantic Communication for Video	2312.05062	已解析
Robust Semantic Communication for Text	2206.02596	已解析
SNR-adaptive Deep JSCC	2102.00202	15
Deep JSCC with Semantic Importance	2302.02287	15
A Mathematical Theory of Semantic Communication	2401.13387	已解析
同上（Overview）	2401.14160	已解析

3. 知识库构建链路（API 直传）

3.1 解析状态机

‘POST /kortex/kb/doc/file/list’ 返回每个文件的 ‘state’ :

- '200': 解析成功, chunk 可检索;
- '100': 处理中(排队/解析);
- '500': 失败(DKE 实例 status=4)。

3.2 上传链路（绕开 UI 文件对话框）

平台前端"导入"对话框使用原生文件选择器,浏览器自动化无法注入文件;

逆向前端`app-pc-kb.js`后得到完整 API 链，已封装为`scripts/kb\_upload.py`：

```
1) GET /aihub/api/v1/sts?type=upload&path=kortex/kb/doc/file/<■■■■>&fileId=<hash>
   → {accessKeyId, accessKeySecret, securityToken, bucket, endpoint,
      type:"MINIO", path:"apsara/kortex/kb/doc/file/<■■■>/<■■■■>"}
   ■■■■ type=upload■■■■■ accessKeySecret■
```

```
2) ■ AWS SigV4 ■ PUT /minio-test/<path> ■■■■■■■■■■ 200■
```

3) POST /kortex/kb/doc/file/uploadFromPreUploadPaths
 {kbCode, paths:[<path>], userMetadata:{}} → ■■■■■■■■

关键坑：

- 上传路径必须用 STS 返回的 `path`（服务端会插入随机目录），不能用本地文件名；
- `userMetadata` 对应前端 sessionStorage 的 `annotateMetadata`，不传即为空；
- 单文件限制 50MB，解析为异步任务，需轮询 `file/list` 的状态。

3.3 chunk 结构

POST /kortex/kb/doc/chunk/list {kbCode, fileCode, current, pageSize} 返回 chunk :

```
{
  "file_code": "...", "file_name": "1809.01733.pdf",
  "chunk_content": "Deep Joint Source-Channel Coding ...",
  "chunk_bboxes": "[{\"page\":1,\"text_bbox\":[...]\"text_content\":...\", \"text_type\": \"paraTitle\"}]"
}
```

'chunk bboxs' 保留页码与版面坐标, 是"引用可溯源到页码"的数据基础。

4. Agent 工作流 (B 路线)

4.1 流程图结构 (11 节点 10 边)



4.3 引用校验节点 (Agent 化改造, v1.4)

‘meta’脚本节点在渲染引用列表的同时执行程序化校验：

- 容错解析结果总结 JSON（去 markdown 围栏、截取首个 JSON 对象）；
- 断言每个引用 `idx` 真实存在于本次召回 chunk（`all\_meta`）中；
- 断言答案正文每个 **【X】** 标记都对应一个 refs 条目；
- 任一不通过 → 输出 "△ 引用校验：[...] 未能与本次召回内容核实"。

同时 `结果总结` prompt 强制"引用 idx 必须真实存在于参考内容中，禁止编造引用"。

> 平台发布纪律：ONLINE 配置不可编辑，只能 `cloneAndCreateNewVersion`（`configCleanSourceEnum=KEEP`）

> 克隆新版本 → 改 DRAFT 流程 → saveProcess → online。v1.0→v1.4 迭代即按此流程。

4.2 修复过程记录（重要排障案例）

原始流程配置了"术语库检索（NewAutoTerm）/ 问答库检索（NewAutoQa）"分支，但知识库是文档类型：术语分支空转产出垃圾值 "rq"，"综合改写兼容"脚本拿到 "rq" 后 600 秒超时。修复：

- 删除 8 个无效节点与 9 条边；
- 新增"多轮改写 → 文档召回"直连边；
- 修正节点入参映射：`文档召回.query = NODE.多轮改写.content`、`结果总结.rewrite\_query = NODE.多轮改写.content`。

流程图保存接口（关键载荷结构）：

```
POST /api/flow/config/saveProcess
{
  "configTargetType": "AGENT",
  "configCode": "<agentCode>_<agentVersion>",
  "version": "<agentVersion>",
  "data": { "nodes": [...], "edges": [...] } // ■■ data ■■■
}
```

读取：`GET /api/flow/config/get?configTargetType=AGENT&configCode;=\_`。

5. 运行时链路（调试/评测自动化）

- 1) POST /designer/createRunningSession {agentCode, agentVersion}
→ data.uniqueCode■■■ ID■
- 2) ■■ SSE■GET /designer/createCmdChannelNew?sessionId=<uniqueCode>&clientId=<cid>
■■■■■ Cookie■run ■■■■■■■■■■
- 3) POST /designer/run {sessionId, input, clientId}
- 4) ■■ POST /designer/listTask {sessionId, clientId}
→ ■■■ typeCode■None(LLM)/KNOWLEDGES_DOC(■■)/SCRIPT(■■)
- 5) ■■■■■■GET /designer/pageListLatestMsg?sessionCode=<uniqueCode>&pageIndex=0&pageSize=50
→ list[].blocks[].blockContent■isRightShow=true ■■■■■■
■■■■■■■■ isRightShow=false ■ assistant ■■■

注意：`listTask` 只给中间任务，最终回答要从 `pageListLatestMsg` 取；

SSE 输出可用于实时进度，但体积大（含 thinking 流），不适合直接解析答案。

6. 自主 Agent（A 路线）

- 类型：角色对话（AutoBotV2），引擎 processAgentEngine/ReAct，模型 qwen3.6-plus；
- 配置持久化在 `GET /agent/version/config/query?code=&version;=&versionName;=`；
- 运行入口：`POST /agent/share/create {agentCode, agentVersion}` → shareUuid →
`#/arrange/agentExp?randomCode=`；
- 已验证：可自主组织"语义通信核心思想 + 4 篇文献引用"的完整回答。

平台限制（已验证，非配置问题）：

- 工具可挂载（`POST /task/add`），但执行报 `AGENT\_TASK\_DEFAULT\_FAILED`；
`/tools/debugExecuteApiTool` 500 —— 工具执行沙箱在平台侧故障；
- 脚本沙箱仅支持 `re/json/string/math/random/set/frozenset/DateTime/uuid`，禁网；
- 角色对话设计页二次访问白屏（前端 bug），配置需在创建后首次访问完成。

7. 评测结果

演示问题集脚本化运行（`scripts/run\_demo\_qa.py`），逐题真实调用 B 路线 Agent，记录回答、耗时与召回来源，结果见 `demo/results/`。

7.1 实测数据（2026-08-23，6/6 全部成功）

#	问题	耗时	召回来源数	回答长度
1	什么是语义通信？与传统通信区别	92s	6	597 字
2	Deep JSCC 核心思想与工作原理	143s	2	723 字
3	Deep JSCC 与 DeepSC 区别	114s	3	630 字
4	语义通信主要挑战	102s	5	649 字
5	任务导向通信与经典范式差异	92s	4	826 字
6	SNR 自适应深度 JSCC 实现	114s	3	865 字

7.2 能力结论

- 文档召回命中新补论文 chunk（例：Q3 召回 2302.02287 等 3 篇，score 0.67-0.76）；
- 最终回答结构化、带 **【I】 - 【V】** 引用且附来源说明；
- 引用可精确到论文章节（例：Q6 回答定位到《SNR-Adaptive Deep JSCC》3.1 节的自适应解码器设计）；
- 单题端到端耗时约 100-180s（多轮改写 + 召回 + 总结）。

7.3 确定性 eval（v1.4，2026-08-24）

`scripts/eval\_agent.py` 三场景断言结果（含平台抖动自动重试一次）：

场景	结果	关键断言
concept	PASS 91.9s	回答存在 / 引用↔refs 一致 / 校验干净
multistep	PASS 132.8s	同上

